

Maschinelles Lernen

8. Faltungsnetze

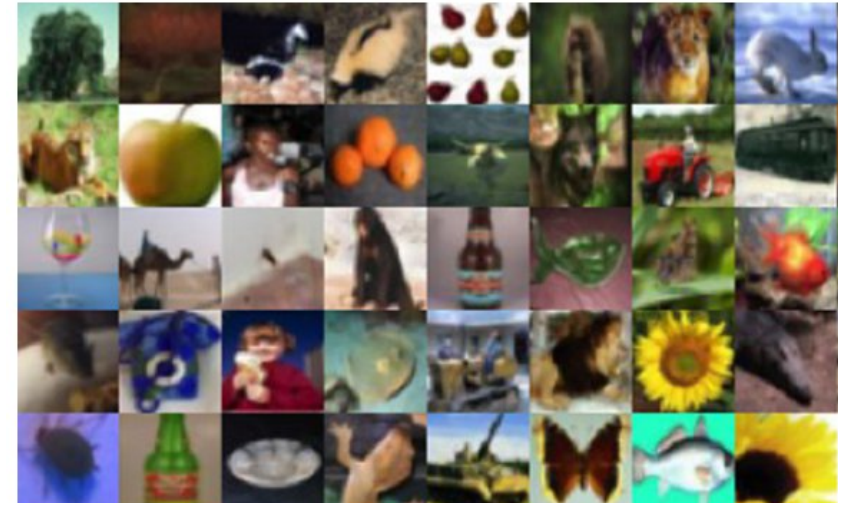
Prof. Dr. Dennis Janka

Hochschule Karlsruhe
University of
Applied Sciences

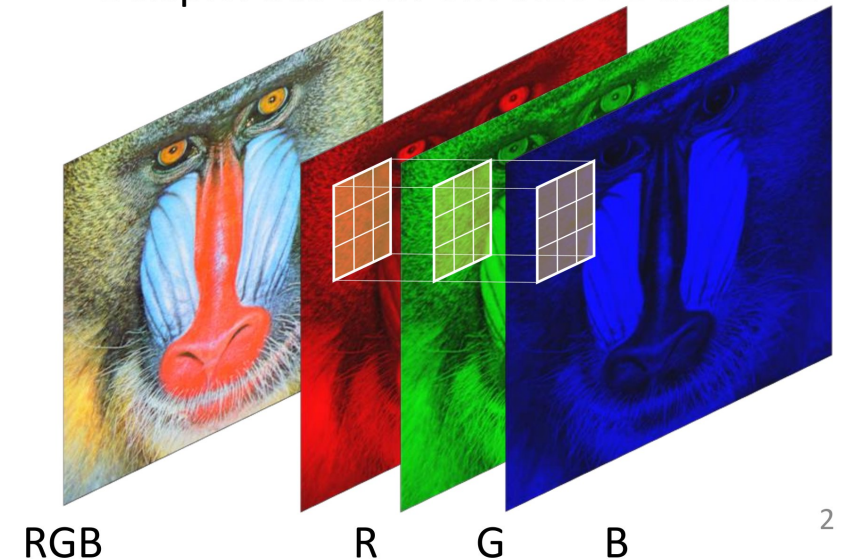


Motivation: Klassifikation von Bildern

- CIFAR100 Datensatz:
 - 60,000 Bilder
 - 20 Superklassen (z.B. aquatische Säugetiere)
 - 5 Klassen pro Superklasse (Biber, Delphin, Otter, Seehund, Wal)
- Jedes Bild hat 32x32 Pixel und drei Kanäle (rot, grün, blau)
- Darstellung eines Bildes als dreidimensionaler Array, sog. **Feature-Map** (ein Element (Tensor) im $\mathbb{R}^{32 \times 32 \times 3}$)

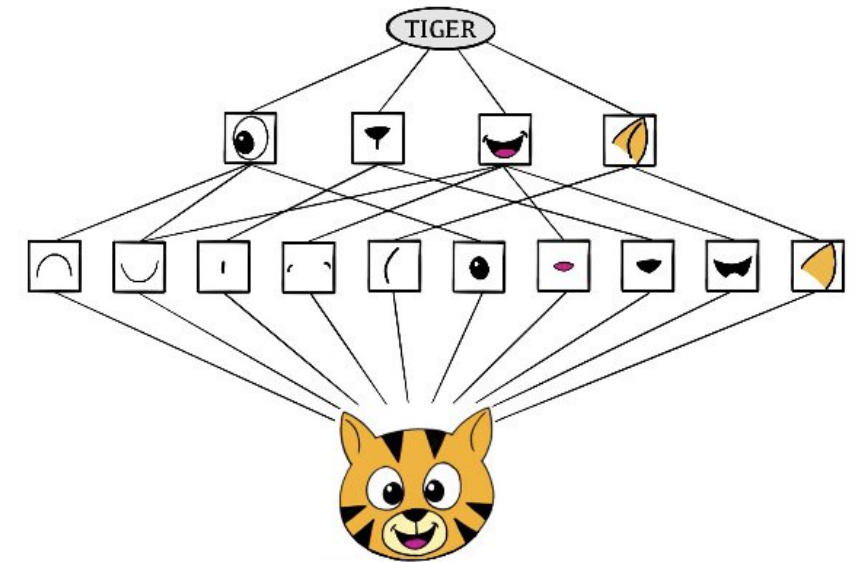


Beispiel aus dem CIFAR100 Datensatz



Faltungsnetze

- **Faltungsnetze** (convolutional neural networks; ConvNets) sind sehr erfolgreich bei der Klassifizierung von Bildern aber auch anderen Aufgaben des maschinellen Sehens.
- Idee: Lerne zu erkennen, ob bestimmte „Features“ (Muster), die bei der Klassifizierung helfen, irgendwo im Bild vorhanden sind.
- Beispiel: Erkenne zunächst low-level Features wie z.B. bestimmte Kanten oder farbige Flecken. Kombiniere diese dann zu abstrakteren Features wie Augen oder Ohren.
- Letzte Schicht: An-/Abwesenheit dieser Features trägt zur Vorhersage der Klasse bei.



Faltungsnetze

Wie wird solch eine Hierarchie gelernt?

- Kombination aus **Faltungsschichten** und **Pooling-Schichten**
- **Faltungsschichten** (conv layers): Suche nach Instanzen von kleinen Mustern im Bild. Welche Muster für die Klassifikation hilfreich sind, wird praktischerweise gleich mit gelernt!
- **Pooling-Schichten** (pooling layers): Aggregieren der Faltungsergebnisse

Faltung

- **Faltung** (convolution): Tiefes mathematisches Konzept (—> digitale Signalverarbeitung).
- Beispiel aus der digitalen Bildverarbeitung (Graustufen-Bild, also Matrix):



Bild (100x100)

$$\star \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} =$$

Faltungskern* (3x3)



Featuremap

- Das Ergebnis der Faltung nennt man auch **Featuremap**.
- Im Beispiel: Die Faltung (auch: **Filter**) transformiert das Bild in eine Version, bei der die Umrisse hervorgehoben sind.

*: auch Faltungsmatrix, Kern, Filterkern, Filteroperator, oder Filtermaske genannt, englisch (convolution) kernel

Faltung



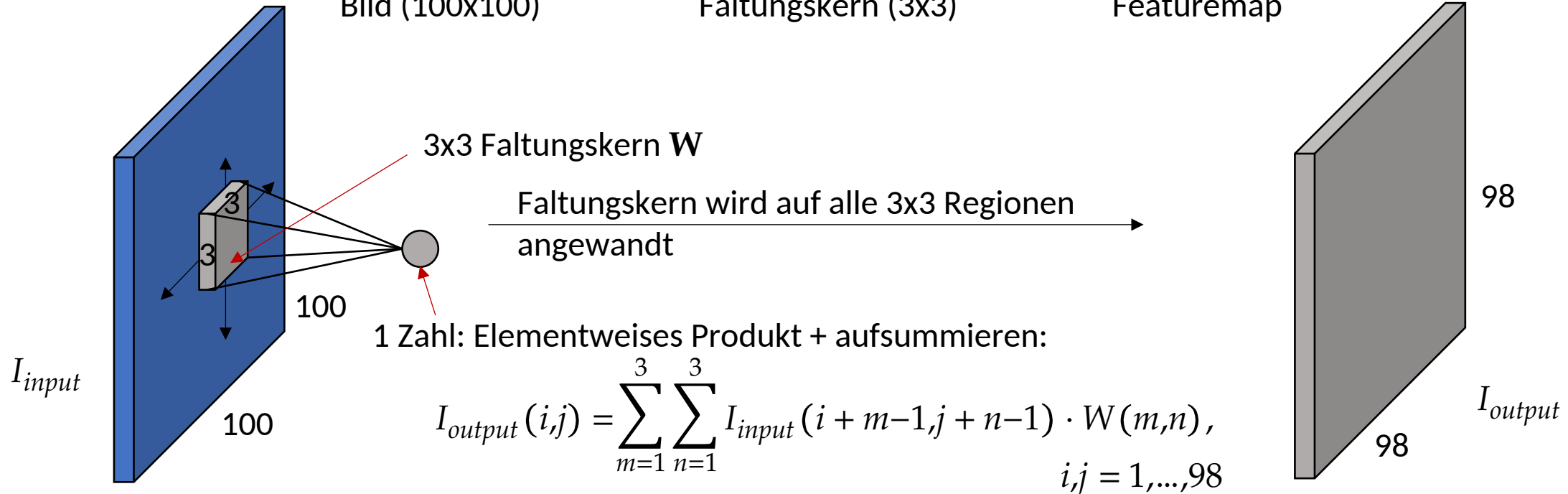
Bild (100x100)

$$\star \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} =$$

Faltungskern (3x3)



Featuremap



Wie können Faltungen Features erkennen?

- Ein Faltungskern der Größe $k \times k$ wird auf **alle** $k \times k$ **Submatrizen** des originalen Bildes angewendet (Faltung).
- Wenn eine $k \times k$ Submatrix des Bildes ähnlich dem Faltungskern ist, hat das gefaltete Bild an der Stelle einen großen Wert, anderenfalls einen kleinen Wert.
- Das heißt, im gefalteten Bild sind die Regionen des originalen Bildes hervorgehoben, die dem Faltungskern ähneln.

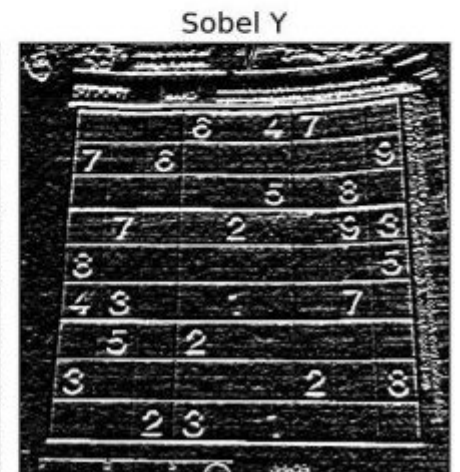
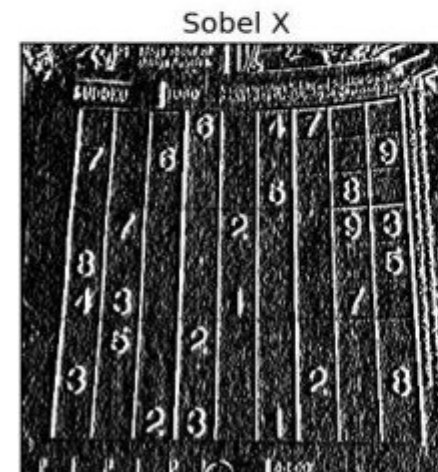


Vertikales Kantenelement

$$\text{Sobel } X = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Horizontales Kantenelement

$$\text{Sobel } Y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$



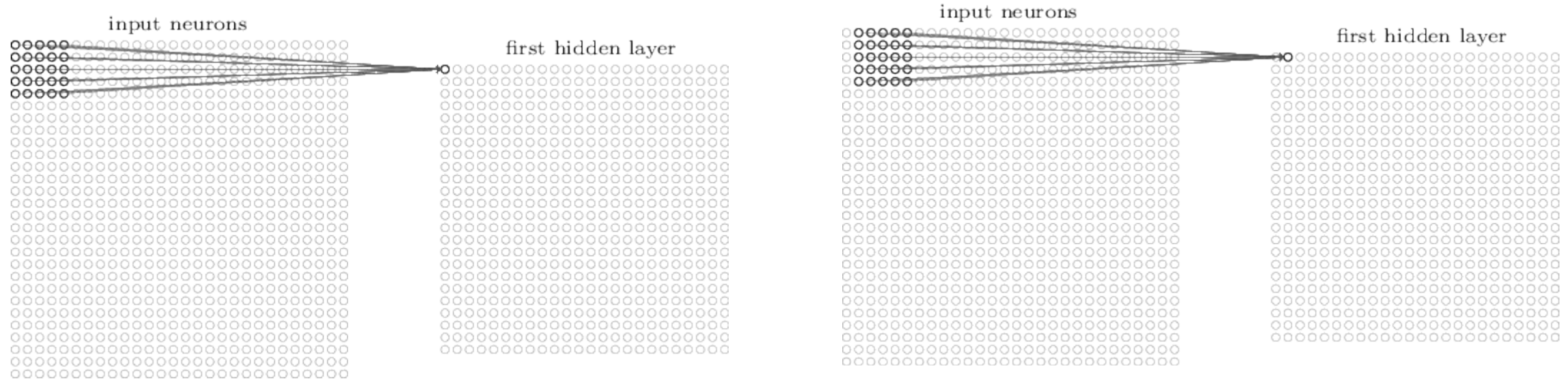
https://docs.opencv.org/4.x/d5/d0f/tutorial_py_gradients.html

Idee Faltungsschicht in neuronalen Netzen

- Frage: Welche Faltungsoperatoren sollen wir auf die Bilder anwenden?
- Antwort: Die, die bei der Klassifikation helfen! Wir **lernen** die Faltungsmatrix **W**, die auf das gesamte Bild angewendet wird (+ bias und Aktivierungsfunktion)



Wie passt das in unser bisheriges Framework von neuronalen Netzen?



Beispiel 5x5 Faltungsmatrix:

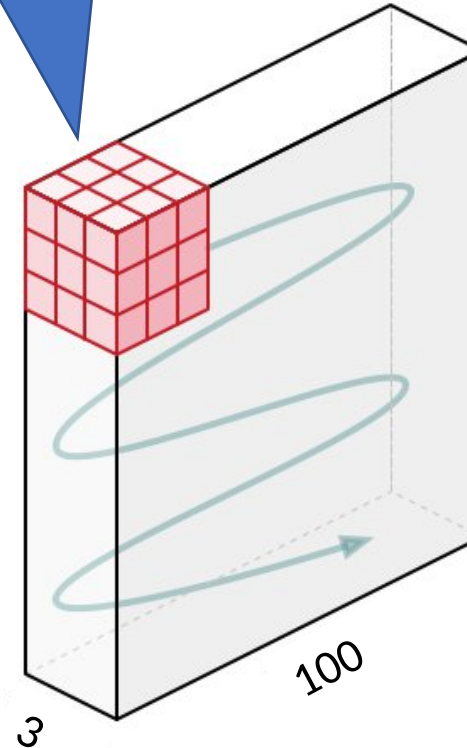
- Jedes Neuron der verdeckten Schicht ist mit 25 Neuronen der Eingangsschicht verbunden
- Jedes Neuron der verdeckten Schicht hat dieselben 25+1 Gewichte („weight sharing“)
- Nach der Faltungsoperation wird eine Aktivierungsfunktion angewendet (typischerweise ReLU)
- Insgesamt erhält man eine Schicht mit viel weniger Parametern!
- Ableitungen können dann wie gewohnt mittels Backpropagation berechnet werden.

Mit drei Kanälen

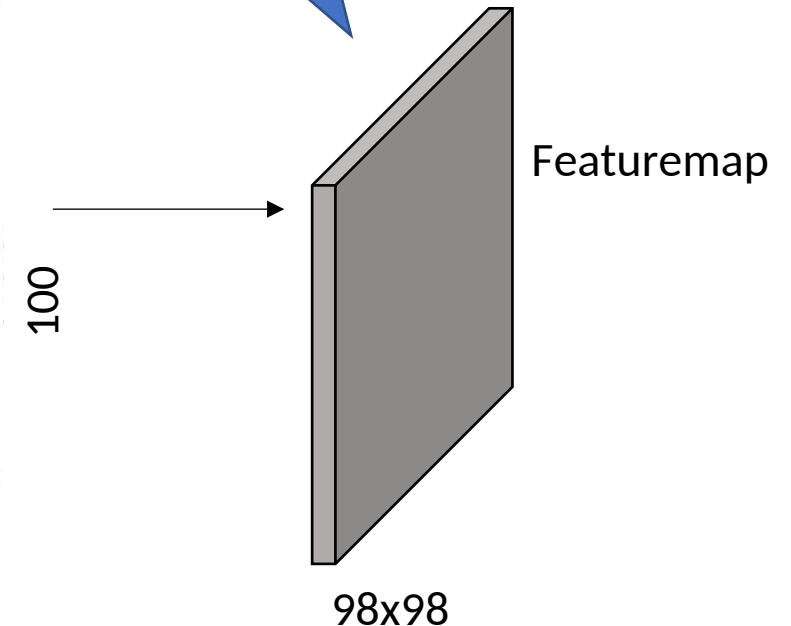
- Bei Farbbildern (RGB) hat der Filterkern auch 3 Kanäle (im Beispiel: ein 3x3 Filterkern für jeden Kanal → 27 Parameter)
- Das Resultat ist eine 2D-Feature Map – es wird über alle drei Dimensionen summiert (aber nicht gefaltet, d.h. der 3x3x3 Filter bewegt sich nur entlang zweier Dimensionen):

$$I_{output}(i,j) = \sum_{c=1}^3 \sum_{m=1}^3 \sum_{n=1}^3 I_{input}(i+m-1, j+n-1, c) \cdot \mathbf{W}(m,n, c), \quad i,j = 1, \dots, 98$$

Tiefe von Eingabedaten
und Filter muss
übereinstimmen!

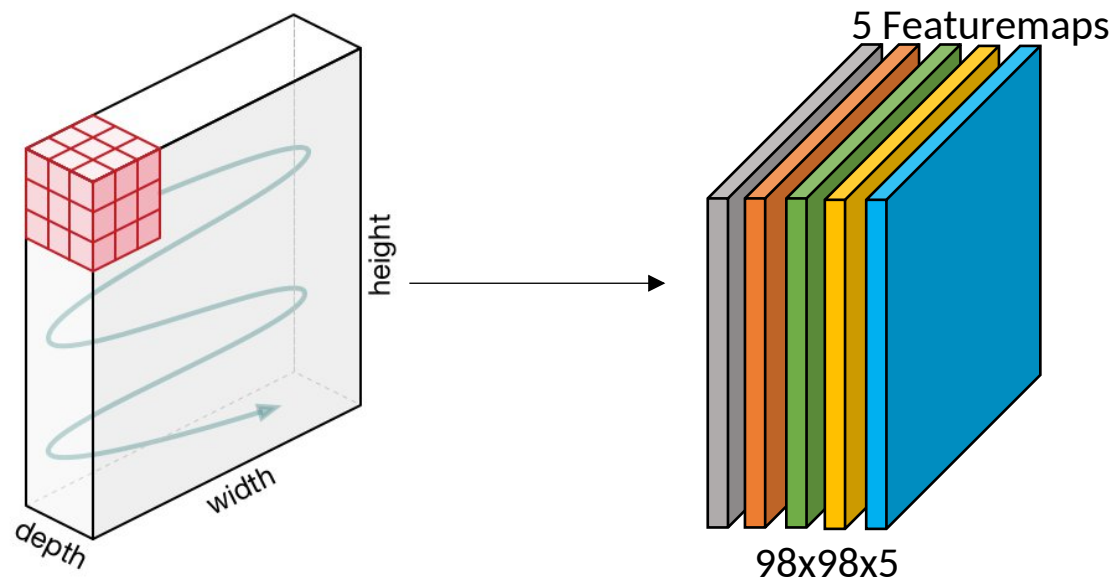


Die Farbinformation (3 Kanäle)
ist nach der Faltung nicht mehr
explizit vorhanden!



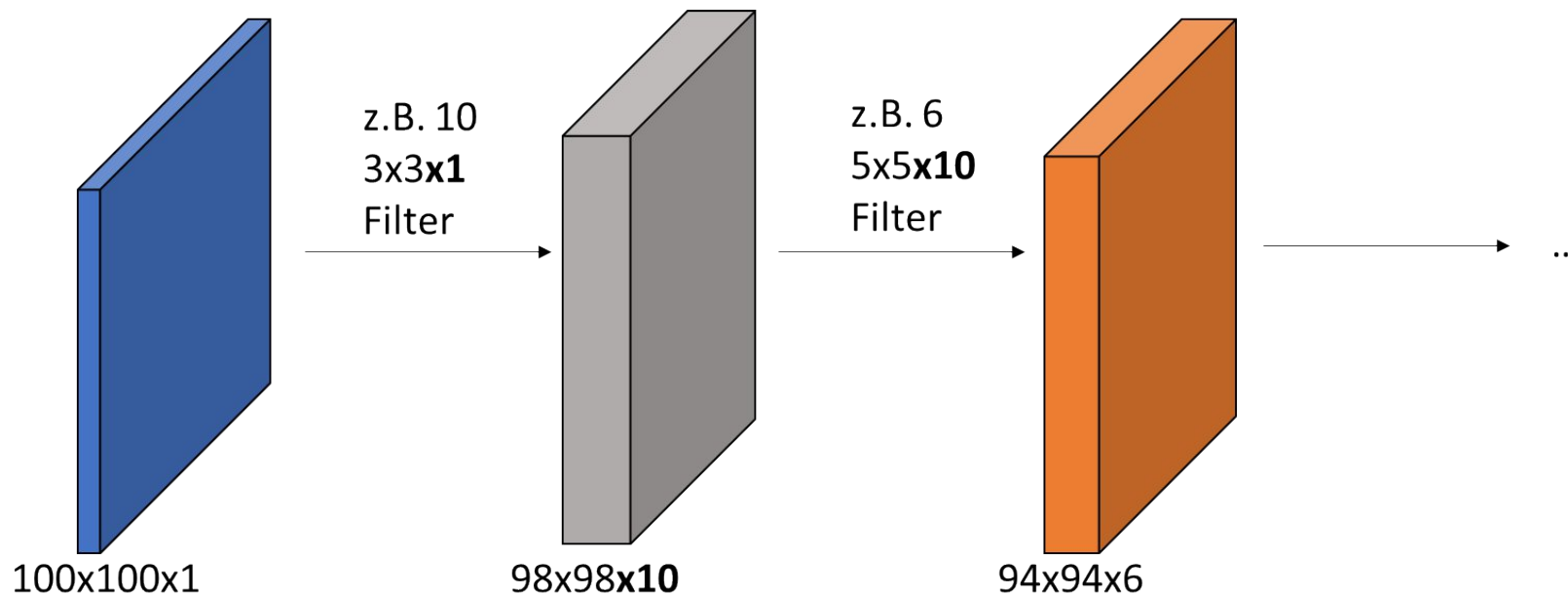
Mehr Filter

- In der Regel möchte man verschiedene Filter trainieren, die unterschiedliche Features erkennen
- Dazu führt man einfach mehrere Filter ein, von denen jeder eine Featuremap produziert
- Anzahl der Filter: Hyperparameter (ähnlich wie Anzahl Neuronen in einer Schicht)



Mehrere Filter hintereinander

Man kann Faltungsschichten also als Abbildungen 3D Tensor $\rightarrow$ 3D Tensor betrachten
Damit ist auch klar, wie man Faltungsschichten hintereinanderschaltet:

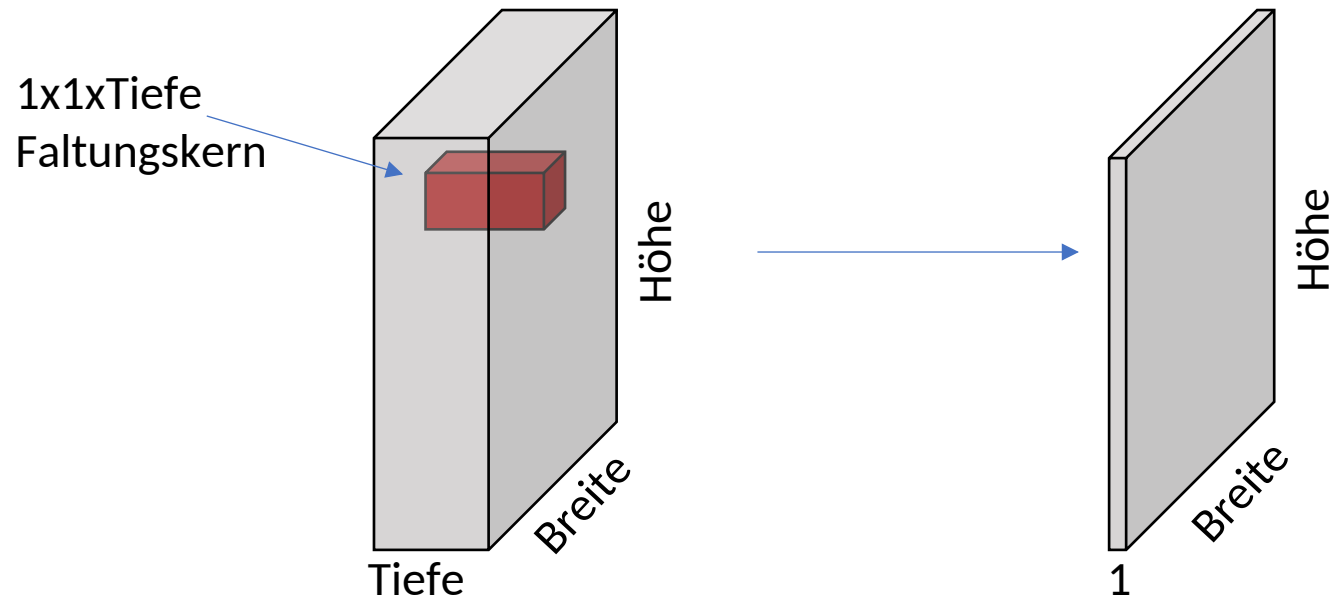


Ein Filter verwendet immer *alle* Eingangskanäle.

Es können $1, \dots, K$ Filter verwendet werden, um $1, \dots, K$ Featuremaps zu generieren.

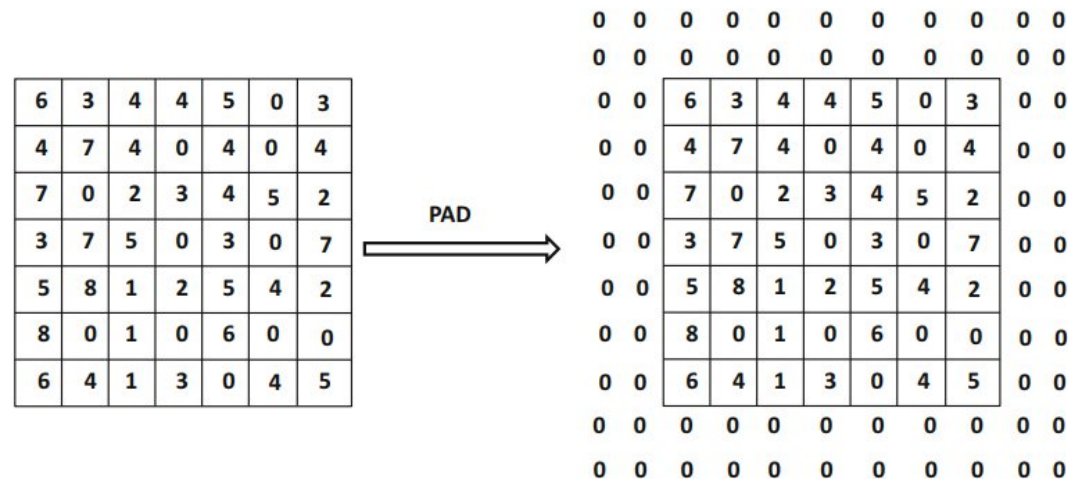
Hyperparameter: Filtergröße

- I.A. kann eine beliebige $m \times n$ -Matrix als Filterkern verwendet werden
- Gängige Werte sind ungerade und quadratische, z.B. 3x3, 5x5 oder 7x7.
- Auch 1x1 Faltungen sind üblich. So können mehrere Featuremaps zu einer einzigen aggregiert werden



Hyperparameter: Padding

- Beobachtung: Faltung reduziert Dimension (Höhe / Breite) des Eingangsbildes um Filtergröße -1
- Durch **Padding** (Auffüllen) mit Nullen kann man (bei ungerader Größe des Faltungskerns) erreichen, dass Ein- und Ausgang der Faltungsschicht die gleiche Höhe und Breite besitzen (sog. half-padding)



- Es verhindert außerdem, dass Pixel am Rand des Bildes in der Featuremap unterrepräsentiert sind, da sie sonst an weniger Faltungsoperationen teilnehmen.

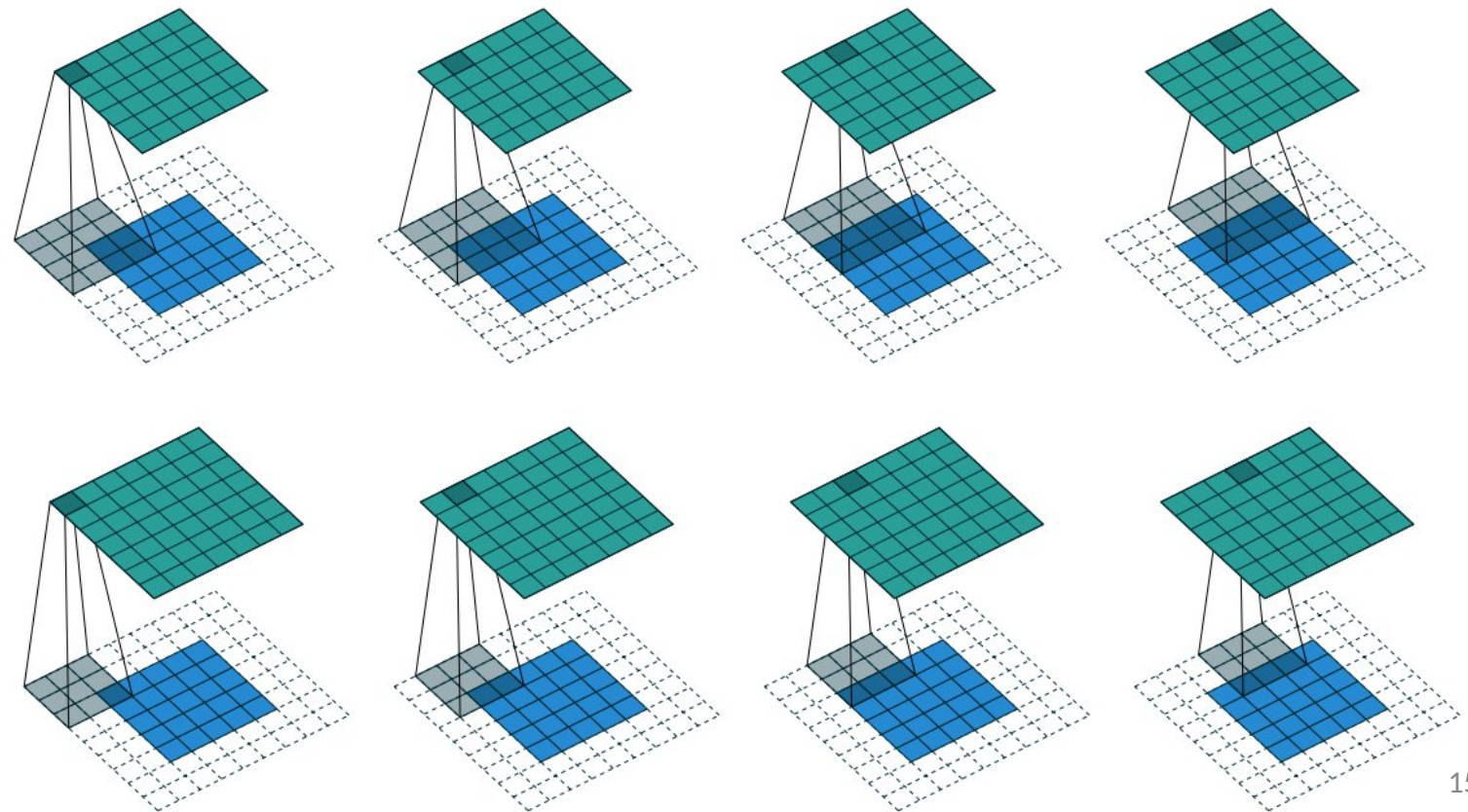
Hyperparameter: Padding

Achtung: bei **gerader** Kerngröße (oberes Bild) oder beim **Full padding** (unteres Bild) hat der Output (grün) eine höhere Dimension als der Input (blau)

Eine detaillierte Beschreibung der Arithmetik von Faltungen findet sich hier:

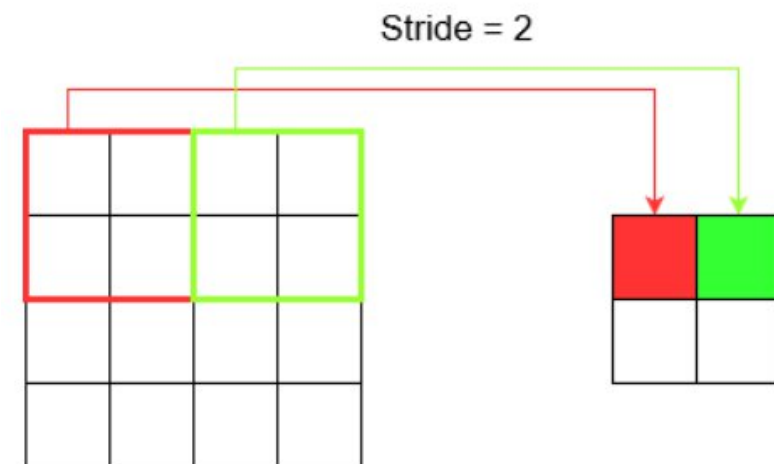
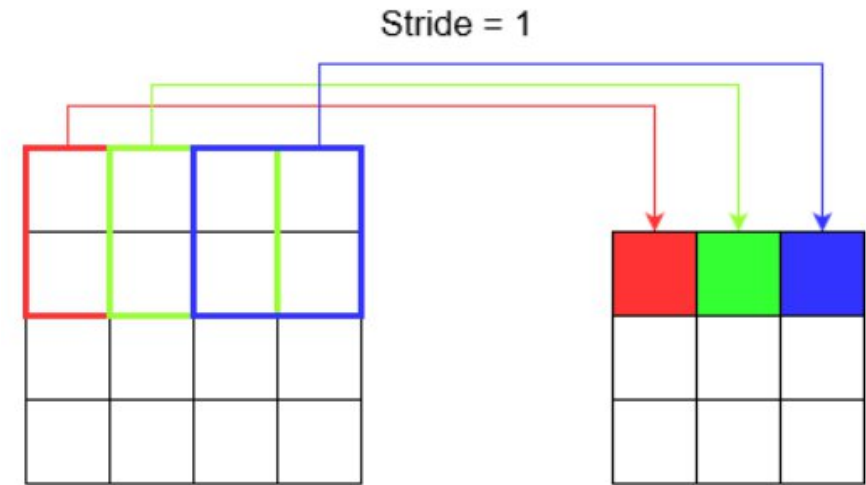
https://github.com/vdumoulin/conv_arithmetic

Aus: Vincent Dumoulin, Francesco Visin: "A guide to convolution arithmetic for deep learning",
<https://arxiv.org/abs/1603.07285>



Hyperparameter: Stride

- Stride (Verschiebungsweite) gibt an, um wie viele Pixel die Filtermatrix „weiterbewegt“ wird.
- Ein höherer Stride führt zu einer niedrigeren Dimension des Outputs, da nicht alle möglichen Submatrizen des Inputs abgetastet werden (Erkennung wird ungenauer/Information geht verloren)
- Üblich ist meist ein Stride von 1.

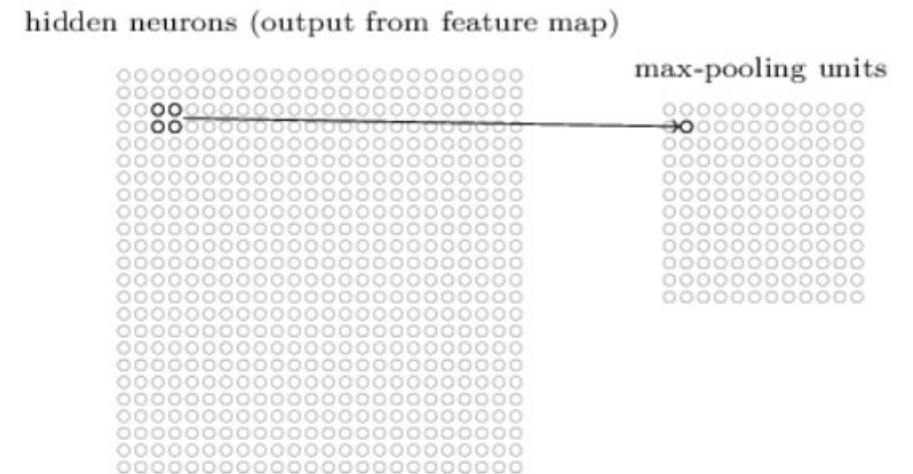


Zusammenfassung Faltungsschichten (conv layer)

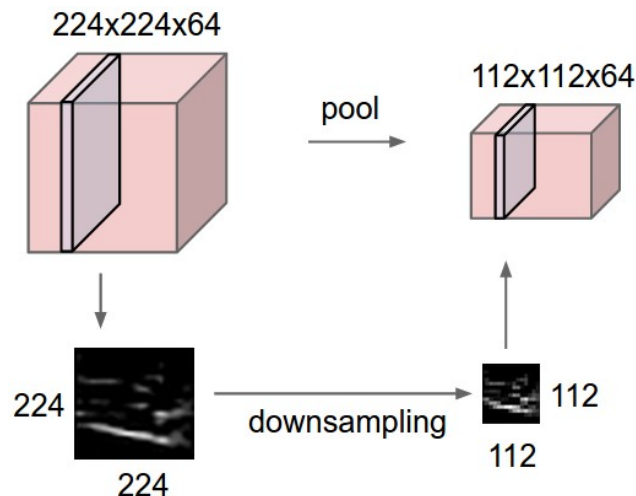
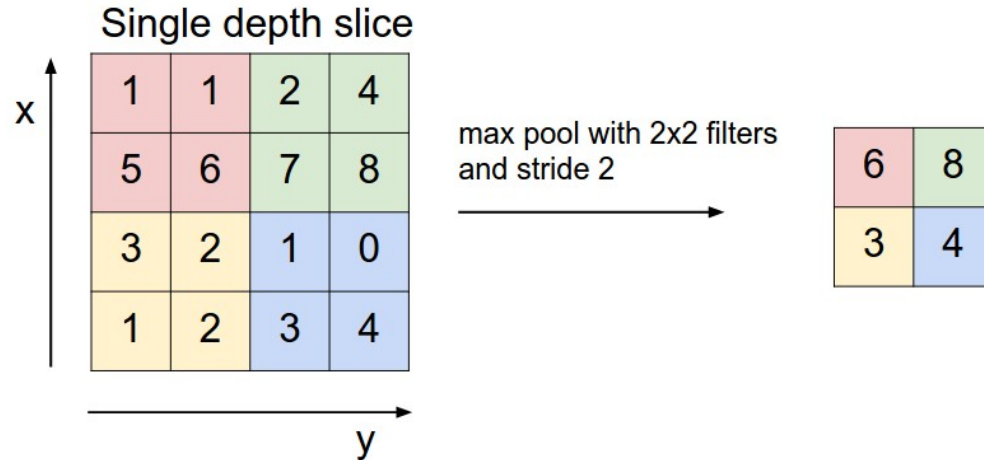
- **Eingabe:** Tensor der Größe $W_{in} \times H_{in} \times D_{in}$
 - **Fünf (bzw. vier) Hyperparameter:**
 - Anzahl Filterkerne K
 - Tensorgröße der Filterkerne F : Tensor der Größe $W_K \times H_K \times D_{in}$ (meist $W_K = H_K$)
 - Verschiebungsweite Stride S
 - Größe des null-aufgefüllten Bereichs auf jeder Seite (zero-padding) P
 - **Ergebnis:** Tensor der Größe $W_{out} \times H_{out} \times D_{out}$:
 - $W_{out} = \left\lfloor \frac{W_{in} - W_K + 2P}{S} \right\rfloor + 1$
 - $H_{out} = \left\lfloor \frac{H_{in} - H_K + 2P}{S} \right\rfloor + 1$
 - $D_{out} = K$
 - **Parameter:** Anzahl insgesamt $K \cdot (W_K \times H_K \times D_{in} + 1)$ ($W_K \times H_K \times D_{in}$ und ein Bias-Wert Gewicht pro Filterkern).
-  $\lfloor \cdot \rfloor$: Abrunden zur nächsten ganzen Zahl

Pooling

- Pooling-Schichten werden üblicherweise Faltungsschichten nachgeschaltet und fassen Informationen der Featuremaps zusammen
- Funktionsweise: Für eine Region, z.B. 2x2 Pixel, wird das Maximum ausgegeben.
- Pooling erkennt, ob ein Feature irgendwo in einer Region des Bildes vorhanden ist, exakte Position wird verworfen
- Idee: Wenn ein Feature gefunden wurde, reicht es für die Klassifikation, die ungefähre Position zu kennen
- Ergebnis: kleinere Featuremap -> weniger Parameter in späteren Schichten!



Pooling-Schicht



- **Eingabe:** Tensor der Größe $W_{in} \times H_{in} \times D_{in}$
- **Hyperparameter:**
 - Räumliche Ausdehnung F
 - Stride S (typischerweise >1)
- **Ergebnis:** Tensor der Größe $W_{out} \times H_{out} \times D_{out}$ mit
 - $W_{out} = \frac{(W_{in}-F)}{S} + 1$
 - $H_{out} = \frac{(H_{in}-F)}{S} + 1$
 - $D_{out} = D_{in}$
- Üblicherweise kein Zero-padding der Eingabe
- Keine zusätzlichen Parameter

Typische Architekturen von Faltungsnetzen

- Wir haben gesehen, wie die einzelnen Schichten funktionieren.
- Diese können im Prinzip beliebig kombiniert und hintereinandergeschaltet werden.
- In der Praxis trifft man oft folgendes Architekturmuster ($0 \leq N \leq 3$, $0 \leq K \leq 2$):

INPUT -> [[CONV -> RELU]*N -> POOL?]*M -> [FC -> RELU]*K -> FC

0..N
Faltungsschichten
gefolgt von ReLU
Aktivierung

Optional: Pooling-
Schicht nach N
Faltungsschichten

0..M mal
wiederholen

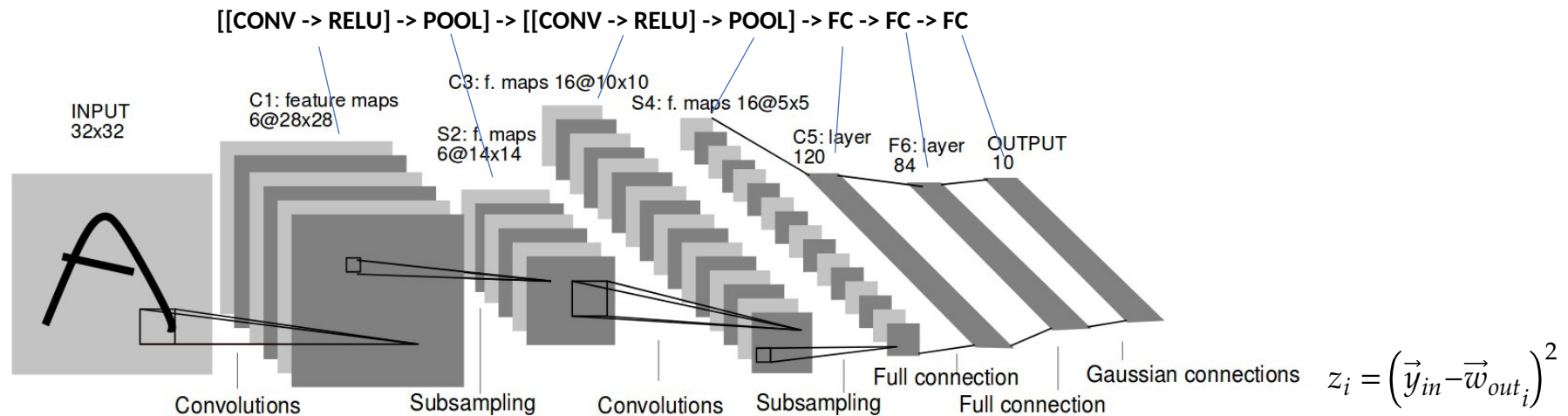
FC = Fully Connected
Schicht: Das sind die
„normalen“ Feedforward
Schichten

Am Ende immer eine FC
Schicht, die auf die
Outputklassen abbildet

Beispiel-Architektur: LeNet

INPUT -> [[CONV -> RELU]*N -> POOL?]*M -> [FC -> RELU]*K -> FC

LeNet: Handschriften-Erkennung N=1, M=2, K=2



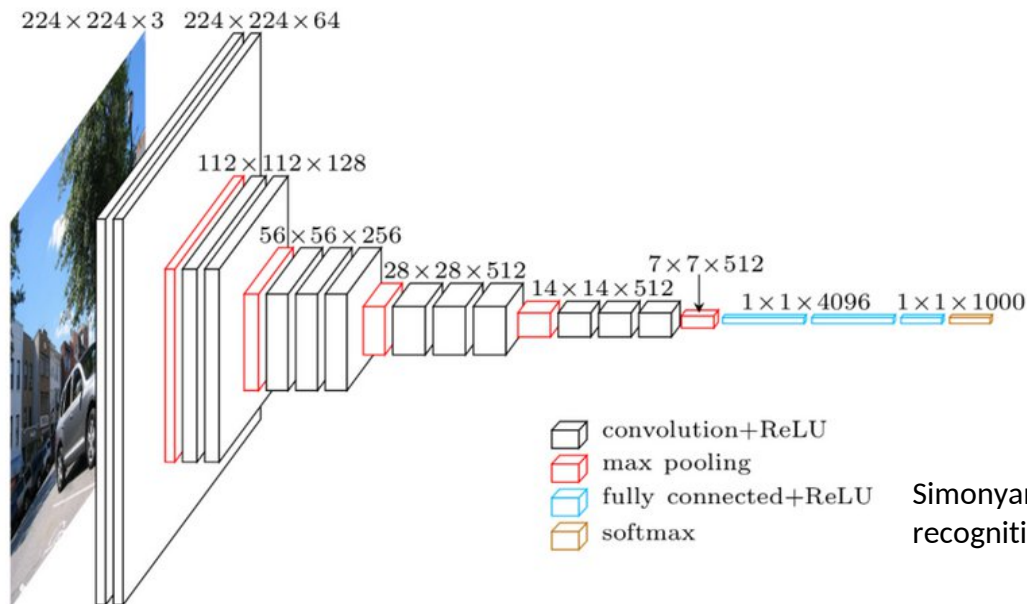
LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278-2324.

Beispiel-Architektur: VGGNet

INPUT $\rightarrow$ $[[\text{CONV} \rightarrow \text{RELU}]^N \rightarrow \text{POOL?}]^M \rightarrow [\text{FC} \rightarrow \text{RELU}]^K \rightarrow \text{FC}$

VGG16: Bild-Klassifikation:

$[[\text{CONV} \rightarrow \text{RELU}]^2 \rightarrow \text{POOL}]^2 \rightarrow [[\text{CONV} \rightarrow \text{RELU}]^3 \rightarrow \text{POOL}]^3 \rightarrow [\text{FC} \rightarrow \text{RELU}]^3 \rightarrow \text{SOFTMAX}$



Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.

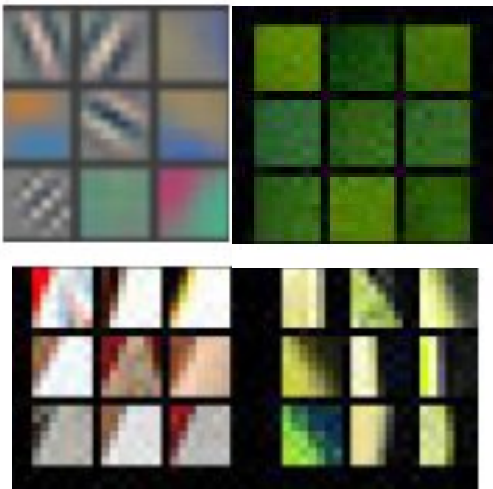
MNIST Beispiel mit Conv Layers

Siehe Jupyter Notebook: „mnist_example_conv.ipynb“ in ILIAS

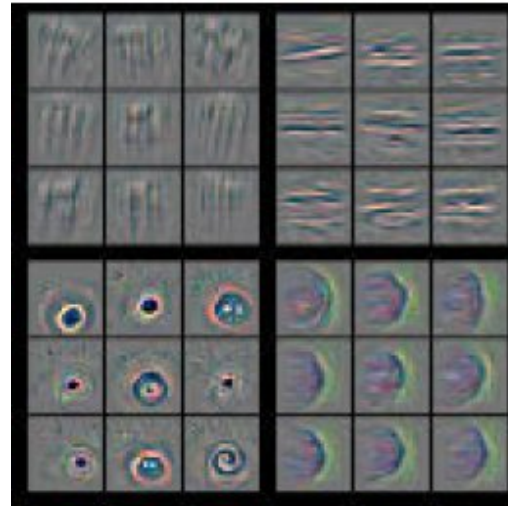
Was passiert in den einzelnen Schichten?

Beispiel: Rückprojektion von jeweils vier Feature Maps einer Schicht in den Pixelraum für jeweils neun Beispiele mit besonders starker Aktivierung (=große Werte).

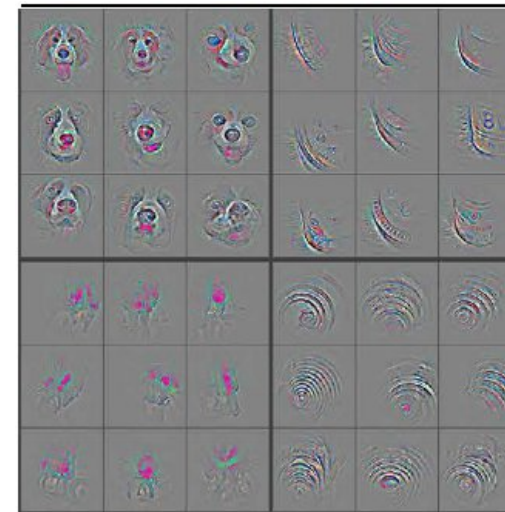
1. Schicht



2. Schicht



4. Schicht



Low-level
feature

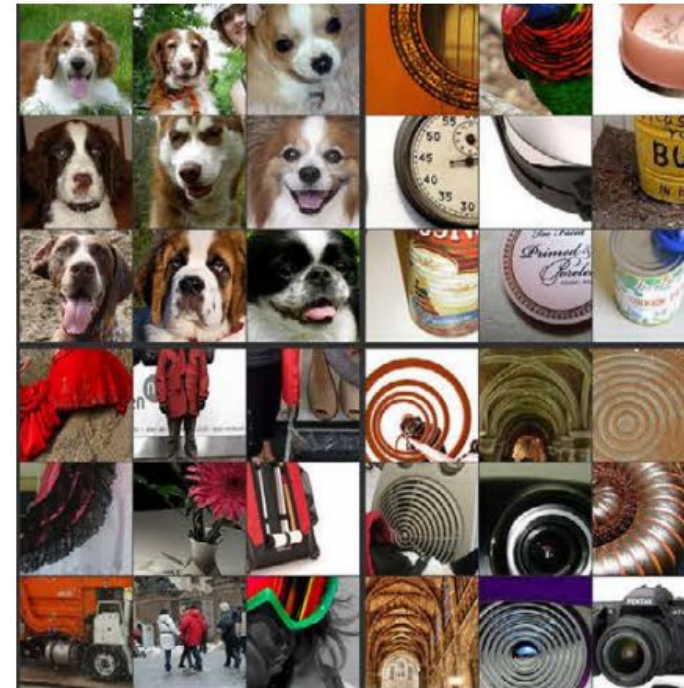
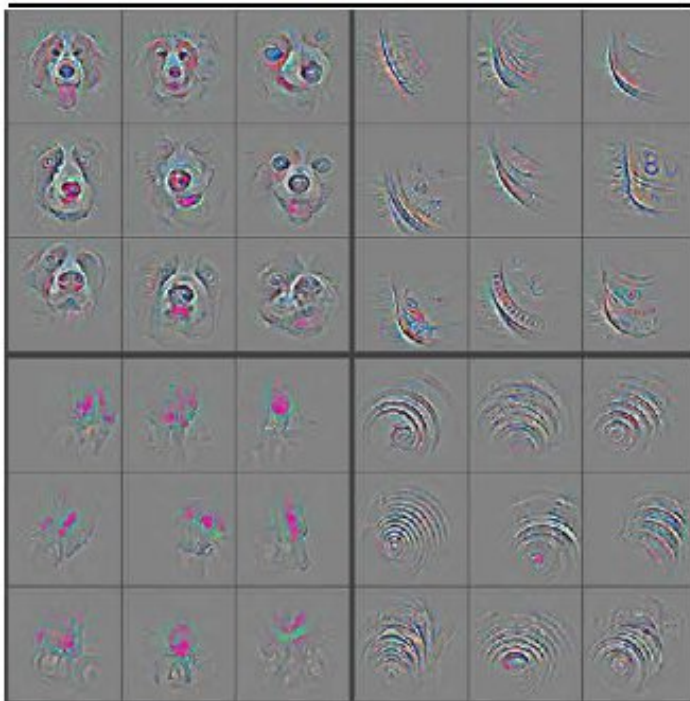
Mid-level
feature

High-level
feature

Klassifikator

Was passiert in den einzelnen Schichten?

- Das Bild links beantwortet die Frage: Welche Teile des Originalbildes haben dazu geführt, dass die Aktivierung in dieser Featuremap so hoch war?



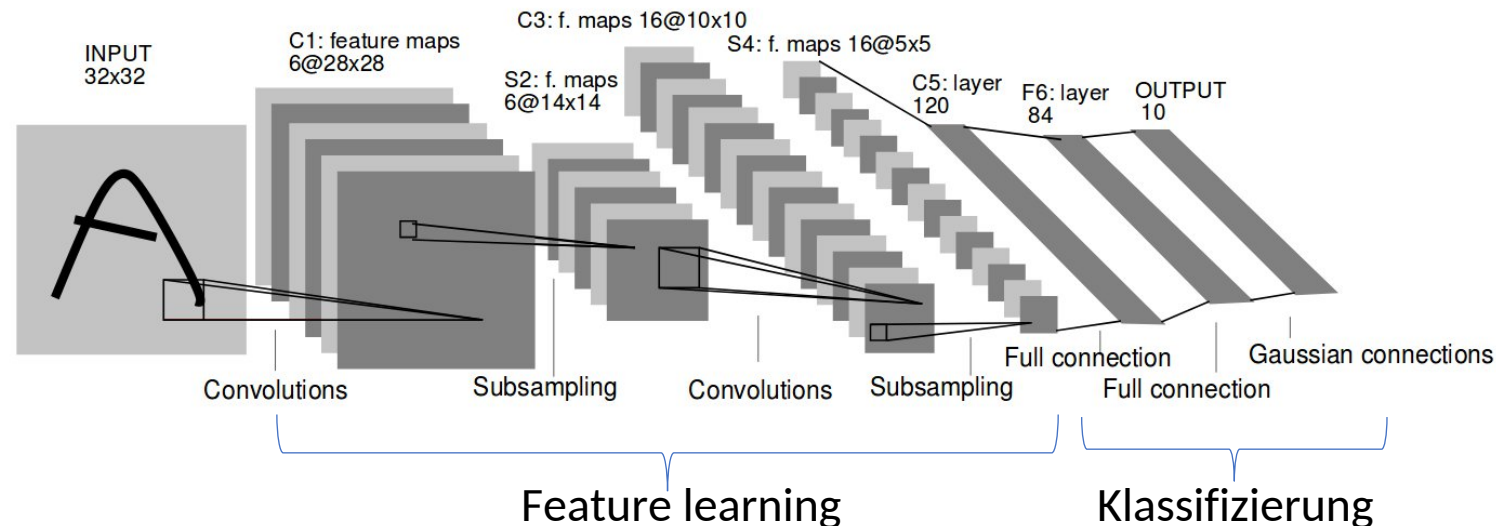
Zugehörige Originalausschnitte der Bilder

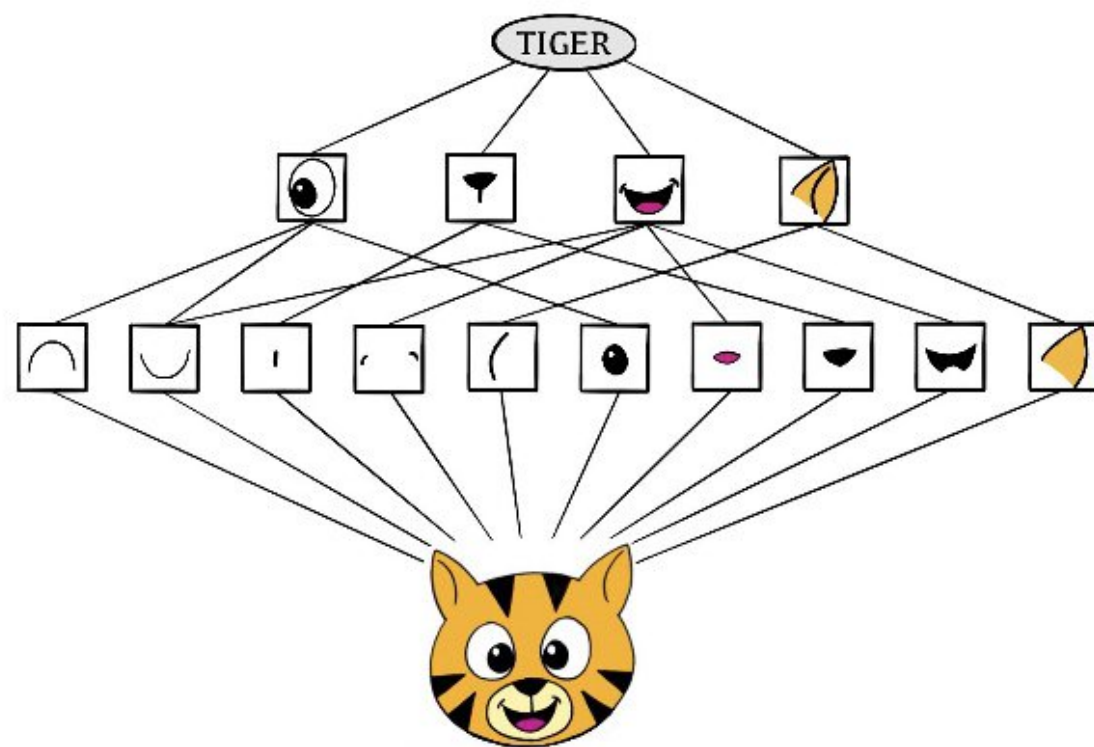
Interpretation als Feature Learning

Zusammenfassend:

In den Faltungsschichten werden hierarchisch Features gelernt („wo im Bild befinden sich Kanten / geometrische Figuren / komplexe Muster“).

In den Feedforward-Schichten werden diese Features gewichtet um die Klasse zu bestimmen (ähnlich wie bei einem „normalen“ NN oder bei einem linearen Modell).





Conv1D / Conv3D